

# Intelligent Software Engineering

## Requirements Engineering

Zhilei Ren



Dalian University of Technology

October 7, 2025



# Outline

- 1 Introduction
- 2 The Challenge of RE & The Rise of LLMs
- 3 Core Prompt Engineering Techniques for RE
- 4 Modeling Requirements as Code with PlantUML
- 5 Practical Application: AI-Assisted Requirements Generation
- 6 Best Practices for AI-Integrated RE
- 7 Conclusion and Future Directions
- 8 Techniques for Requirements Engineering Research (Bonus)



# Definition of Requirements Engineering

Requirements engineering is an interdisciplinary function that mediates between the domains of the acquirer and supplier or developer to establish and maintain the requirements to be met by the system, software or service of interest. Requirements engineering is concerned with discovering, eliciting, developing, analyzing, verifying (including verification methods and strategy), validating, communicating, documenting and managing requirements<sup>1</sup>.

---

<sup>1</sup>ISO/IEC/IEEE 29148 Systems and software engineering —Life cycle processes – Requirements engineering ▶



# As Proposed by the Project Sponsor



# As Specified in the Project Request



# As Designed by the Senior Systems Analyst



# As Produced by the Programmers



# As Installed at the User's Site

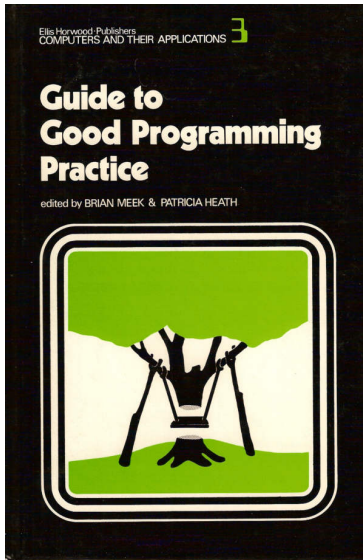




# What The User Wanted



# Guide to Good Programming Practice, 1979



# Bug or Feature?

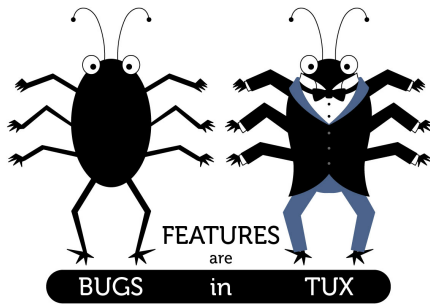


Figure 1: Features = Bugs in Tux



# The First Computer Bug

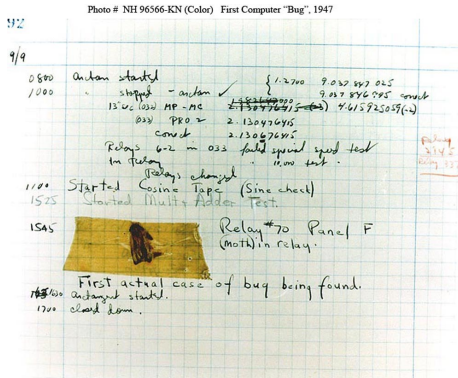


Figure 2: The First Computer Bug



# The First Computer Bug

Menlo Park mch 3<sup>rd</sup> 78

W<sup>m</sup> Oulton Esq.

Dear Sir

You were partly correct, I did find a "bug" in my apparatus, but it was not in the telephone proper. It was of the genus "callbellum". The insect appears to find conditions for its existence in all call apparatus of telephones. Another delay was the sickness of Adam's wife. I intend to present you with a first class phonograph for your home, for reproducing music etc. this apparatus will run with a clockwork train. I will also place one in the room called "experimental room" if you will be so kind as to inform me where that is.

I wish you could find time some afternoon to come down and see my experimental room, (no desks manned with mathematicians) and hear some good phonographic singing and talking.

Yours Truly  
Jno A Edison



# Combo in Fighting Games was originally a Bug

Combos were a design accident; lead producer Noritaka Funamizu noticed that extra strikes were possible during a bug check on the car-smashing bonus stage. He thought that the timing required was too difficult to make it a useful game feature, but left it in as a hidden one<sup>2</sup>.



Figure 4: Street Fighter 2

<sup>2</sup>[https://en.wikipedia.org/wiki/Combo\\_\(video\\_games\)](https://en.wikipedia.org/wiki/Combo_(video_games))



# Hidden Features aka Easter Egg



Figure 5: The Konami Code

Finding the game too difficult to play through during testing, he (Konami developer) created the cheat code, which gives the player a full set of power-ups... The code was meant to be removed prior to publishing, but this was overlooked and only discovered as the game was being prepared for mass production. The developers decided to leave it there, as removing it could result in new bugs and glitches<sup>3</sup>.

<sup>3</sup>[https://en.wikipedia.org/wiki/Konami\\_Code](https://en.wikipedia.org/wiki/Konami_Code)



# Hidden Features aka Easter Egg



Figure 5: The Konami Code



Finding the game too difficult to play through during testing, he (Konami developer) created the cheat code, which gives the player a full set of power-ups... The code was meant to be removed prior to publishing, but this was overlooked and only discovered as the game was being prepared for mass production. The developers decided to leave it there, as removing it could result in new bugs and glitches<sup>3</sup>.

<sup>3</sup>[https://en.wikipedia.org/wiki/Konami\\_Code](https://en.wikipedia.org/wiki/Konami_Code)





# Be Careful with Putting Easter Egg in Software

Why does man print "gimme gimme gimme" at 00:30?

Asked 7 years, 10 months ago Modified 30 days ago Viewed 563k times

2011

We've noticed that some of our automatic tests fail when they run at 00:30 but work fine the rest of the day. They fail with the message

```
gimme gimme gimme
```

in `stderr`, which wasn't expected. Why are we getting this output?

date man

Share Improve this question Follow

edited Jul 15, 2021 at 10:55  
Admin AdminBee  
user 23.6k ● 25 ● 54 ● 77

asked Nov 20, 2017 at 14:19  
Jaroslav Kucera  
10.8k ● 5 ● 17 ● 30

(a) Test Failure due to man

2672

Dear @colmmacauli, I think that if you type "man" at 0001 hours it should print "gimme gimme gimme". #abba  
[@mananet - 3 November 2011](#)

er, that was my fault, I suggested it. Sorry.

Pretty much the whole story is in the commit. The maintainer of man is a good friend of mine, and one day six years ago I jokingly said to him that if you invoke man after midnight it should print "gimme gimme gimme", because of the Abba song called "Gimme gimme gimme a man after midnight".

Well, he did actually [put it in](#). A few people were amused to discover it, and we mostly forgot about it until today.

(b) The Root Cause

Figure 6: Bug Caused by an Easter Egg



# Outline

- 1 Introduction
- 2 The Challenge of RE & The Rise of LLMs**
- 3 Core Prompt Engineering Techniques for RE
- 4 Modeling Requirements as Code with PlantUML
- 5 Practical Application: AI-Assisted Requirements Generation
- 6 Best Practices for AI-Integrated RE
- 7 Conclusion and Future Directions
- 8 Techniques for Requirements Engineering Research (Bonus)



# Traditional RE Challenges

## Common Issues:

- Incomplete requirements
- Changing requirements
- Communication gaps
- Ambiguous specifications
- Stakeholder conflicts

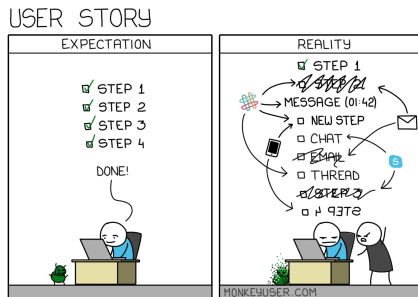


Figure 7: Expectation vs. Reality



# AI-Assisted Requirements Engineering

## The New Paradigm

Using prompt engineering techniques to enhance traditional requirements engineering processes

- **Automated Requirements Generation:** AI can help draft initial requirements
- **Requirements Analysis:** AI can identify inconsistencies and gaps
- **Use Case Development:** AI can generate and refine use cases
- **Specification Writing:** AI can help create clear, unambiguous specifications
- **Validation Support:** AI can simulate scenarios and identify edge cases



# What is Prompt Engineering?

## Definition

Prompt Engineering is the practice of designing and optimizing inputs (prompts) to effectively communicate with AI models to generate desired outputs.

- **Precision:** Crafting clear, specific instructions
- **Context:** Providing relevant background information
- **Structure:** Organizing information logically
- **Iteration:** Refining based on model responses
- **Validation:** Ensuring output meets requirements



# Parallels Between RE and Prompt Engineering

| Requirements Engineering    | Prompt Engineering               |
|-----------------------------|----------------------------------|
| Stakeholder needs analysis  | Understanding user intent        |
| Requirements elicitation    | Prompt design and formulation    |
| Use case modeling           | Scenario specification           |
| Requirements specification  | Clear instruction crafting       |
| Validation and verification | Output evaluation and refinement |
| Change management           | Prompt iteration                 |

Table 1: Comparative Analysis



# Outline

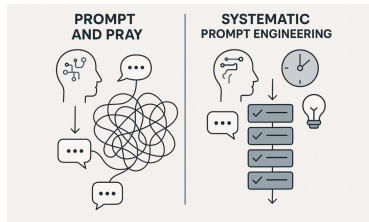
- 1 Introduction
- 2 The Challenge of RE & The Rise of LLMs
- 3 Core Prompt Engineering Techniques for RE**
- 4 Modeling Requirements as Code with PlantUML
- 5 Practical Application: AI-Assisted Requirements Generation
- 6 Best Practices for AI-Integrated RE
- 7 Conclusion and Future Directions
- 8 Techniques for Requirements Engineering Research (Bonus)



# Prompt Engineering Techniques

## Key Techniques:

- Zero-shot vs. Few-shot prompting
- Chain-of-Thought reasoning
- Role-playing scenarios
- Template-based prompts
- Iterative refinement





# Zero-shot vs. Few-shot Prompting

## Zero-shot Prompting

Providing no examples - the model must understand and respond based solely on the instruction

## RE Example - Zero-shot

```
"Generate functional requirements for a user login system"
```



# Zero-shot vs. Few-shot Prompting

## Few-shot Prompting

Providing several examples to demonstrate the desired pattern or format

### RE Example - Few-shot

"Here are examples of good requirements:

1. The system shall validate user credentials
2. The system shall encrypt passwords

Now generate requirements for user registration"



# Zero-shot vs. Few-shot: Practical Examples

## Zero-shot Example:

```
Create a use case for
"password reset" functionality
```

## Few-shot Example:

```
Example 1: Use case for login
- Actor: User
- Precondition: Valid account
- Steps: Enter credentials...
```

```
Example 2: Use case for logout
- Actor: User
- Precondition: Logged in
- Steps: Click logout...
```

```
Now create for "password reset"
```

### Pros:

- Quick and simple
- Good for straightforward tasks
- Less context needed

### Pros:

- Better quality output
- Consistent formatting
- Handles complex patterns



# Chain-of-Thought Reasoning

## Definition

Breaking down complex problems into intermediate reasoning steps, mimicking human problem-solving

## RE Application

Instead of asking for complete requirements, guide the AI through logical steps:

- 1 Identify stakeholders
- 2 Define high-level goals
- 3 Break down into features
- 4 Specify detailed requirements
- 5 Validate completeness



# Chain-of-Thought: Requirements Example

## Benefits for Requirements Engineering:

- Reduces ambiguity in complex domains
- Ensures logical consistency
- Helps identify missing requirements
- Improves traceability

Let's design requirements for an online payment system:

Step 1: Identify main stakeholders

- Customers, merchants, payment processors, banks

Step 2: Define core functionality needed

- Payment processing, security, fraud detection

Step 3: For payment processing, consider:

- Payment methods (credit card, digital wallets)
- Transaction flow (initiation, validation, completion)
- Error handling (declined payments, timeouts)

Step 4: Security requirements:

- Data encryption, PCI compliance, authentication

Step 5: Now generate detailed functional requirements for the payment processing component...

## Listing 1: Chain-of-Thought for Payment System



# Role-playing Scenarios

## Concept

Asking the AI to adopt a specific persona or role to generate more context-appropriate responses

## RE Roles Examples

- **Product Owner:** "Act as a product owner for an e-commerce platform..."
- **System Architect:** "You are a system architect designing a microservices architecture..."
- **Quality Assurance Engineer:** "As a QA engineer, create test scenarios for..."
- **End User:** "From the perspective of a novice user, what features would you need..."

## Advantages:

- Generates perspective-specific requirements
- Identifies conflicting needs early
- Improves requirement completeness



# Role-playing in Requirements Elicitation

## Without Role-playing:

List requirements for a project management tool

## With Role-playing:

Act as a project manager with 10 years experience in agile environments. What requirements would you prioritize for a project management tool?

## Specific Role Examples:

Role: Security Auditor

"Identify security requirements for a healthcare app handling patient data, considering HIPAA compliance"

Role: Accessibility Specialist

"Generate accessibility requirements for a banking application to ensure WCAG 2.1 AA compliance"



# Template-based Prompts

Using structured templates to ensure consistent, comprehensive outputs across different requirements

## Common RE Templates:

- Use Case Template
- User Story Template
- Functional Requirement Template
- Non-functional Requirement Template
- Acceptance Criteria Template

## Benefits

- Standardized format across the project
- Easier validation and review
- Better integration with RE tools
- Consistent quality of generated content



# Template Examples for Requirements Engineering

```
USE CASE TEMPLATE:
[System Name] - [Use Case Name]

ID: [Unique Identifier]
Primary Actor: [Role]
Secondary Actors: [Optional Roles]

Preconditions:
- [Condition 1]
- [Condition 2]

Main Success Scenario:
1. [Step 1]
2. [Step 2]
3. [Step 3]

Alternative Flows:
- [A1]: [Description]
- [A2]: [Description]

Postconditions:
- [State 1]
- [State 2]

Exceptions:
- [E1]: [Error condition]
```

Listing 2: Use Case Template



# Functional Requirements Template

## REQUIREMENT TEMPLATE:

Requirement ID: [FR-001]

Type: [Functional/Non-functional]

Priority: [High/Medium/Low]

### Description:

[Clear, unambiguous description of what the system shall do]

### Source:

[Stakeholder/Business Need/Regulation]

### Rationale:

[Why this requirement is needed]

### Acceptance Criteria:

1. [Criterion 1 - testable condition]
2. [Criterion 2 - testable condition]
3. [Criterion 3 - testable condition]

### Dependencies:

- [Related requirement IDs]

### Constraints:

- [Technical/business constraints]

Status: [Proposed/Approved/Implemented]

## Listing 3: Structured Requirements Template

# Iterative Refinement

Starting with broad requirements and progressively refining them through multiple iterations of feedback and clarification

## Iterative Process:

- 1 **First Pass:** High-level requirements and scope
- 2 **Second Pass:** Detailed functional requirements
- 3 **Third Pass:** Non-functional requirements and constraints
- 4 **Final Pass:** Validation and acceptance criteria

## RE Application

- Start with epic-level user stories
- Break down into features
- Refine into detailed user stories
- Add technical specifications
- Validate with stakeholders

# Iterative Refinement Example

## Iteration 1: High-level

Create requirements for  
user authentication system

## Iteration 2: More specific

Add multi-factor  
authentication support

## Iteration 3: Detailed

Specify timeout policies  
and password complexity rules

## Iteration 4: Validation

Add error handling for  
failed authentication attempts

## Benefits of Iterative Approach:

- Catches ambiguities early
- Allows for stakeholder feedback at each stage
- Reduces rework
- Builds comprehensive requirement sets



# Complete Iterative Refinement Workflow

```
// Stage 1: Scope Definition
"Define the high-level scope for a customer relationship management system"

// Stage 2: Feature Identification
"Based on the scope, identify key features needed for lead management"

// Stage 3: Detailed Requirements
"For the lead management feature, specify functional requirements including lead capture,
assignment, and tracking"

// Stage 4: Validation Criteria
"Add acceptance criteria and test scenarios for lead assignment functionality"

// Stage 5: Edge Cases
"Identify and specify requirements for edge cases like duplicate lead detection and lead
expiration"
```

Listing 4: Multi-stage Refinement Process



# Technique Selection Guide

| Technique            | Best For                                     | RE Phase                          |
|----------------------|----------------------------------------------|-----------------------------------|
| Zero-shot            | Simple, well-understood requirements         | Initial scoping                   |
| Few-shot             | Consistent formatting, complex patterns      | Detailed specification            |
| Chain-of-Thought     | Complex domains, logical decomposition       | Analysis and modeling             |
| Role-playing         | Stakeholder perspective, conflict resolution | Elicitation and validation        |
| Template-based       | Standardization, tool integration            | Specification and documentation   |
| Iterative refinement | Complex systems, evolving requirements       | All phases, especially management |

**Table 2:** Selecting the Right Prompt Engineering Technique

## Combination Approach:

- Start with role-playing for stakeholder perspective
- Use chain-of-thought for complex analysis
- Apply templates for consistent documentation
- Iterate based on feedback



# Case Study: Applying Multiple Techniques

## Scenario: Healthcare Patient Portal

Developing requirements for a secure patient portal with medical records access

### Applied Techniques:

- 1 **Role-playing:** "Act as a healthcare compliance officer concerned with HIPAA"
- 2 **Chain-of-Thought:** Break down security requirements step by step
- 3 **Template-based:** Use standardized requirement templates
- 4 **Few-shot:** Provide examples of good medical software requirements
- 5 **Iterative refinement:** Multiple rounds of stakeholder review



# Summary: Prompt Engineering for RE

## Key Takeaways:

- Different techniques serve different RE needs
- Techniques can and should be combined
- Start simple, add complexity as needed
- Always validate AI-generated requirements
- Document your prompt strategies for reproducibility

## Next Steps:

- Practice each technique with sample RE scenarios
- Develop organizational prompt templates
- Establish validation processes for AI-generated content
- Integrate with existing RE workflows





# Outline

- 1 Introduction
- 2 The Challenge of RE & The Rise of LLMs
- 3 Core Prompt Engineering Techniques for RE
- 4 Modeling Requirements as Code with PlantUML**
- 5 Practical Application: AI-Assisted Requirements Generation
- 6 Best Practices for AI-Integrated RE
- 7 Conclusion and Future Directions
- 8 Techniques for Requirements Engineering Research (Bonus)



# What is PlantUML?

- An open-source tool to create diagrams from plain text language.
- **Core Idea:** Describe the diagram in a human-readable text file, and PlantUML generates the image.
- **Philosophy:** Focus on content and relationships, not manual layout.

**Text** → **PlantUML** → **Diagram**



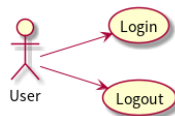
# Why PlantUML for AI-Assisted RE?

- **Text-to-Diagram:** LLMs excel at generating structured text. PlantUML is a perfect output format for them.
- **Version Control:** Prompts and generated '.puml' files can be versioned with Git, tracking the evolution of requirements.
- **Automation:** Generated diagrams can be automatically included in documentation pipelines (like this presentation).
- **Iterative Refinement:** It's easy to ask an LLM to modify a text file. This is much harder with binary image formats.



# Syntax Example: Use Case Diagram

```
1 @startuml
2 left to right direction
3 actor User
4 usecase "Login" as UC_Login
5 usecase "Logout" as UC_Logout
6
7 User --> UC_Login
8 User --> UC_Logout
9 @enduml
```



- Simple, declarative syntax.
- Define actors and use cases.
- Connect them with arrows.



# Outline

- 1 Introduction
- 2 The Challenge of RE & The Rise of LLMs
- 3 Core Prompt Engineering Techniques for RE
- 4 Modeling Requirements as Code with PlantUML
- 5 Practical Application: AI-Assisted Requirements Generation**
- 6 Best Practices for AI-Integrated RE
- 7 Conclusion and Future Directions
- 8 Techniques for Requirements Engineering Research (Bonus)



# Part 1: Prompting for a Use Case Diagram

## The Prompt

Act as a requirements engineer. Generate a use case diagram for an online banking system including: authentication, balance check, funds transfer, and transaction history. Use PlantUML syntax.

- **Role:** "Act as a requirements engineer"
- **Context:** "online banking system"
- **Task:** "Generate a use case diagram"
- **Format:** "Use PlantUML syntax"



# The Generated PlantUML Code

```

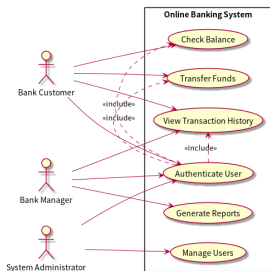
1 @startuml
2 left to right direction
3 actor "Bank Customer" as Customer
4
5 rectangle "Online Banking System" {
6     usecase "Authenticate User" as UC1
7     usecase "Check Balance" as UC2
8     usecase "Transfer Funds" as UC3
9     usecase "View Transaction History" as UC4
10 }
11 Customer --> UC1
12 Customer --> UC2
13 Customer --> UC3
14 Customer --> UC4
15
16 UC2 .-> UC1 : <<include>>
17 UC3 .-> UC1 : <<include>>
18 UC4 .-> UC1 : <<include>>
19 @enduml

```

**Listing 5:** Online Banking System Use Case Diagram



# The Final Diagram



## Key Components:

- Actor: Bank Customer
- System Boundary
- Use Cases: Authenticate, Check Balance, etc.
- Relationships: Includes, Associations





## Part 2: Prompting for a Use Case Specification

### The Follow-up Prompt

Expand the 'Transfer Funds' use case with:  
primary actor, preconditions, main success scenario,  
alternative flows, and postconditions. Format as a  
structured use case specification.

**This is an example of iterative refinement.**



# The Generated Specification

```
Use Case: Transfer Funds
ID: UC-003
Primary Actor: Bank Customer

Preconditions:
1. User is authenticated.
2. User has an active account with sufficient funds.

Main Success Scenario:
1. User selects "Transfer Funds".
2. System displays transfer form.
3. User enters recipient details and amount.
4. System validates details.
5. System processes transfer.
6. System displays confirmation.

Alternative Flows:
A1: Insufficient funds - System shows error.
A2: Invalid recipient - System shows error.

Postconditions:
1. Funds are deducted from source account.
2. Funds are credited to recipient account.
3. A transaction record is created.
```

Listing 6: Transfer Funds Use Case Specification



# Part 3: From Use Cases to Formal Specs

## The Prompt

Convert the 'Transfer Funds' use case into formal functional requirements. Include: ID, description, priority, and acceptance criteria.

**This bridges the gap between user-centric models and system-centric specifications.**



# Generated Functional Requirements

Requirement ID: FR-005

Description: The system shall allow an authenticated user to transfer funds to a valid recipient.

Priority: High

Acceptance Criteria:

1. Given the user is authenticated, when the user selects the transfer option, the system must display the transfer form.
2. Given the transfer form is displayed, the user must be able to input recipient and amount.
3. Given the user submits the transfer, the system must validate that the source account has sufficient funds.
4. Given a valid transfer, the system must debit the source and credit the recipient account.
5. Given the transfer is complete, the system must display a confirmation message.

## Listing 7: Functional Requirements Specification



# Generating Non-Functional Requirements

## The Prompt

```
Generate non-functional requirements for the 'Transfer Funds' feature, focusing on performance and security.
```

```
ID: NFR-001 (Performance)
The system shall process a standard fund transfer in
less than 2 seconds under normal load.

ID: NFR-002 (Security)
All data transmitted during the fund transfer process
must be encrypted using TLS 1.3.

ID: NFR-003 (Security)
Transfers exceeding $1,000 shall require two-factor
authentication.
```

## Listing 8: Non-Functional Requirements



# Outline

- 1 Introduction
- 2 The Challenge of RE & The Rise of LLMs
- 3 Core Prompt Engineering Techniques for RE
- 4 Modeling Requirements as Code with PlantUML
- 5 Practical Application: AI-Assisted Requirements Generation
- 6 Best Practices for AI-Integrated RE**
- 7 Conclusion and Future Directions
- 8 Techniques for Requirements Engineering Research (Bonus)



# Prompt Engineering Patterns for RE

## Effective Patterns:

- Role-playing
- Iterative refinement
- Context provision
- Few-shot examples
- Use structured templates

## Antipatterns to Avoid:

- Vague or ambiguous terms
- Overly complex prompts
- Assuming domain knowledge
- Open-ended questions
- Inconsistent terminology



# An Effective Prompt Template

```
[ROLE] Act as a [specific role, e.g., senior QA engineer]

[CONTEXT] For a [system, e.g., healthcare management app]

[TASK] Generate [artifact, e.g., test cases for the
patient login feature]

[REQUIREMENTS] That cover:
- [Requirement 1, e.g., successful login]
- [Requirement 2, e.g., invalid password]
- [Requirement 3, e.g., locked account]

[CONSTRAINTS] With the following constraints:
- [Constraint 1, e.g., consider HIPAA privacy rules]

[FORMAT] Present the output in [format, e.g., a Gherkin table]
```

Listing 9: Structured Prompt Template for RE





# Human-in-the-Loop is Critical

## Warning

AI-generated content is a starting point, not a final product. It must be critically reviewed, validated, and refined by human engineers.

## Your Role as an Engineer:

- **Validate Correctness:** Does the output meet the actual needs?
- **Ensure Completeness:** Has the AI missed any edge cases or non-functional requirements?
- **Refine for Clarity:** Is the language precise and unambiguous?
- **Take Ownership:** You are ultimately responsible for the requirements, not the AI.

The code and the developer: we're good as long as one of them is able to run



# Human-in-the-Loop is Critical

## Warning

AI-generated content is a starting point, not a final product. It must be critically reviewed, validated, and refined by human engineers.

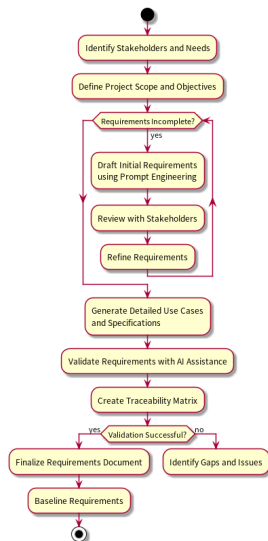
## Your Role as an Engineer:

- **Validate Correctness:** Does the output meet the actual needs?
- **Ensure Completeness:** Has the AI missed any edge cases or non-functional requirements?
- **Refine for Clarity:** Is the language precise and unambiguous?
- **Take Ownership:** You are ultimately responsible for the requirements, not the AI.

The code and the developer: we're good as long as one of them is able to run



# Proposed RE-AI Integration Workflow



# Outline

- 1 Introduction
- 2 The Challenge of RE & The Rise of LLMs
- 3 Core Prompt Engineering Techniques for RE
- 4 Modeling Requirements as Code with PlantUML
- 5 Practical Application: AI-Assisted Requirements Generation
- 6 Best Practices for AI-Integrated RE
- 7 Conclusion and Future Directions**
- 8 Techniques for Requirements Engineering Research (Bonus)



# Key Takeaways

- **Synergy:** Prompt Engineering enhances traditional Requirements Engineering
- **Efficiency:** AI can accelerate requirements development and validation
- **Quality:** Structured prompts lead to better requirements specifications
- **Accessibility:** Visual models (like PlantUML) improve stakeholder understanding
- **Iterative Nature:** Both RE and PE benefit from continuous refinement

## The Future

AI will become an integral partner in requirements engineering, with prompt engineering as the essential interface between human intent and machine capability.

# Outline

- 1 Introduction
- 2 The Challenge of RE & The Rise of LLMs
- 3 Core Prompt Engineering Techniques for RE
- 4 Modeling Requirements as Code with PlantUML
- 5 Practical Application: AI-Assisted Requirements Generation
- 6 Best Practices for AI-Integrated RE
- 7 Conclusion and Future Directions
- 8 Techniques for Requirements Engineering Research (Bonus)



# Next Release Problem

Given:

- A set of software requirements  $R = \{r_1, r_2, \dots, r_n\}$ ,
- A set of customers  $C = \{c_1, c_2, \dots, c_m\}$ ,
- Each customer  $c_j \in C$  requests a subset of requirements  $R_j \subseteq R$  and provides a profit  $p_j > 0$  if all requirements in  $R_j$  are satisfied,
- Each requirement  $r_i \in R$  has an associated cost  $\text{cost}(r_i) > 0$ ,
- A total available budget  $B > 0$ .

The goal is to select a subset of requirements  $R' \subseteq R$  such that:

- 1 The total cost does not exceed the budget:  $\sum_{r_i \in R'} \text{cost}(r_i) \leq B$ ,
- 2 The total profit is maximized:

$$\max_{\substack{R' \subseteq R \\ c_j \in C \\ R_j \subseteq R'}} \sum p_j.$$

This is known as the **Next Release Problem (NRP)** and is a well-known NP-hard problem in requirements engineering and software release



# The Next Release Problem

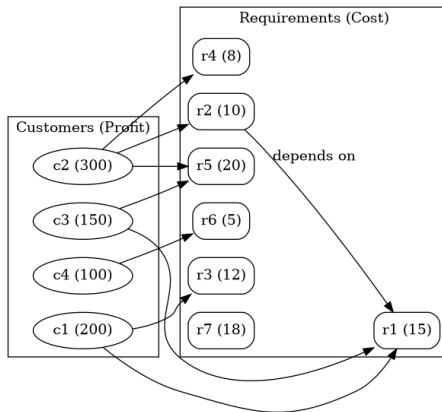


Figure 8: A demo instance





# The Next Release Problem

```

from pyscipopt import Model

# 1. Define problem data
requirements = [f'r{i}' for i in range(1, 8)]
costs = { 'r1': 15, 'r2': 10, 'r3': 12, 'r4': 8, 'r5': 20, 'r6': 5, 'r7': 18 }
BUDGET = 50
customers = {
    'c1': {'reqs': ['r1', 'r3'], 'profit': 200},
    'c2': {'reqs': ['r2', 'r4', 'r5'], 'profit': 300},
    'c3': {'reqs': ['r1', 'r5'], 'profit': 150},
    'c4': {'reqs': ['r6'], 'profit': 100}
}

# 2. Create a SCIP model
model = Model("NRP_Solver")

# 3. Create variables
# Binary variables for each requirement (1 if selected, 0 otherwise)
req_vars = {r: model.addVar(vtype="B", name=r) for r in requirements}
# Binary variables for each customer's satisfaction
cust_sat_vars = {c: model.addVar(vtype="B", name=f'sat_{c}') for c in customers}

# 4. Add Constraints
# Constraint 1: Budget constraint
model.addCons(sum(costs[r] * req_vars[r] for r in requirements) <= BUDGET)
# Constraint 2: Feature Dependencies
# If r2 is selected, r1 must also be selected (r2 -> r1  <=>  r2 <= r1)
model.addCons(req_vars['r2'] <= req_vars['r1'])

```



# The Next Release Problem

```
# Constraint 3: Customer Satisfaction
# A customer is satisfied if all their requested requirements are met.
for name, data in customers.items():
    # This is equivalent to sat_c = AND(reqs), which can be modeled as:
    # sat_c <= req_i for all req_i in data['reqs']
    for req in data['reqs']:
        model.addCons(cust_sat_vars[name] <= req_vars[req])
    # sat_c >= sum(req_i) - (len(reqs) - 1)
    model.addCons(cust_sat_vars[name] >= sum(req_vars[r] for r in data['reqs']) - (len(data['reqs']) - 1))
# 5. Define Objective Function: Maximize total profit
total_profit = sum(data['profit'] * cust_sat_vars[name] for name, data in customers.items())
model.setObjective(total_profit, "maximize")
# 6. Solve the problem
model.optimize()
# 7. Print the solution
if model.getStatus() == "optimal":
    print("Optimal Solution Found:")
    selected_reqs = [r for r in requirements if model.getVal(req_vars[r]) > 0.9]
    final_cost = sum(costs[r] for r in selected_reqs)
    final_profit = model.getObjVal()
    print(f"\nSelected Requirements: {' ', ' '.join(sorted(selected_reqs))}")
    print(f"\nTotal Cost: {final_cost} (Budget: {BUDGET})")
    print("\nCustomer Satisfaction:")
    for name, data in customers.items():
        if model.getVal(cust_sat_vars[name]) > 0.9:
            status = "SATISFIED"
        else:
            status = "NOT SATISFIED"
        req_names = sorted(data['reqs'])
        print(f" - Customer {name} (requests {req_names}): {status}")
```



# References and Further Reading

- [1] Sommerville, I. (2011) [Software Engineering](#)
- [2] Wiegers, K. & Beatty, J. (2013) [Software Requirements](#)
- [3] White, J. et al. (2023) [A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT](#)
- [4] PlantUML Documentation (2024) [Use Case Diagram Guide](#)
- [5] Nuseibeh, B. & Easterbrook, S. (2000) [Requirements Engineering: A Roadmap](#)

**Thank You!**  
Questions?

